

Triggers in SQL

Detailed Notes with Examples

Introduction to Triggers

Definition

A **Trigger** is a special kind of stored procedure that is automatically executed by the database system as a side effect of certain modifications (events) to the database.

Key Requirements for Designing Triggers

- Specify the **event** that causes the trigger to be checked (e.g., insert, update, delete).
- Specify a **condition** that must be satisfied for the trigger to execute.
- Specify the **actions** to be performed when the trigger fires.

Once created, the database system automatically executes the trigger when the specified event occurs and the condition holds true.

Triggering Events and Timing

Trigger Events

- **INSERT** — Trigger fires on insertion of new rows.
- **UPDATE** — Trigger fires on modification of existing rows; can be limited to specific columns.
- **DELETE** — Trigger fires on deletion of rows.

Triggers can be specified to fire **BEFORE** or **AFTER** the event occurs.

Referencing Old and New Values

- **REFERENCING OLD ROW AS orow** — Access row before update/delete.
- **REFERENCING NEW ROW AS nrow** — Access row after insert/update.

Example: Enforcing Integrity Constraints Using Triggers

Example 1: Prevent Insert if Foreign Key Does Not Exist

```
1 CREATE TRIGGER timeslot_check1
2 AFTER INSERT ON section
3 REFERENCING NEW ROW AS nrow
4 FOR EACH ROW
5 WHEN (nrow.time_slot_id NOT IN (
6     SELECT time_slot_id FROM time_slot
7 ))
8 BEGIN
9     ROLLBACK;
10 END;
```

Example 2: Prevent Deletion if Referenced

```
1 CREATE TRIGGER timeslot_check2
2 AFTER DELETE ON time_slot
3 REFERENCING OLD ROW AS orow
4 FOR EACH ROW
5 WHEN (
6     orow.time_slot_id NOT IN (
7         SELECT time_slot_id FROM time_slot
8     ) AND orow.time_slot_id IN (
9         SELECT time_slot_id FROM section
10    )
11 )
12 BEGIN
13     ROLLBACK;
14 END;
```

Before Triggers and Data Modification

Example 3: Convert Blank Grades to NULL

```
1 CREATE TRIGGER setnull
2 BEFORE UPDATE OF grade ON takes
3 REFERENCING NEW ROW AS nrow
4 FOR EACH ROW
5 WHEN (nrow.grade = ' ')
6 BEGIN ATOMIC
7     SET nrow.grade = NULL;
8 END;
```

Trigger to Maintain Derived Data

Example 4: Update credits_earned After Grade Changes

```
1 CREATE TRIGGER credits_earned
2 AFTER UPDATE OF grade ON takes
3 REFERENCING NEW ROW AS nrow
4 REFERENCING OLD ROW AS orow
5 FOR EACH ROW
6 WHEN (
7     nrow.grade <> 'F' AND nrow.grade IS NOT NULL
8     AND (orow.grade = 'F' OR orow.grade IS NULL)
9 )
10 BEGIN ATOMIC
11     UPDATE student
12     SET tot_cred = tot_cred + (
13         SELECT credits FROM course WHERE course.course_id = nrow.
14         course_id
15     )
16     WHERE student.id = nrow.id;
17 END;
```

Statement-Level Triggers

Definition

Statement-level triggers execute **once per SQL statement**, regardless of the number of rows affected.

Use `FOR EACH STATEMENT` instead of `FOR EACH ROW`.

Transition tables (`REFERENCING OLD TABLE, NEW TABLE`) hold all rows affected by the statement.

When to Use Statement-Level Triggers

- More efficient for large batch updates or deletes.
- When the trigger logic relates to the whole statement rather than individual rows.

When Not to Use Triggers

Limitations and Risks of Triggers

- Legacy usage for summary maintenance or replication is now better handled by:
 - Materialized views for summaries.
 - Built-in replication support.
 - Encapsulation with methods or stored procedures.
- Triggers can cause unintended executions, e.g., during data loading or replication.
- Can lead to failures of critical transactions if trigger logic errors occur.
- Possible cascading execution can complicate debugging.

Best Practices

- Disable triggers during bulk data load or replication.
- Keep trigger logic simple and well-tested.
- Avoid triggers for complex application logic better suited for stored procedures.

Summary

Key Points

- Triggers automate actions in response to data changes (insert, update, delete).
- They consist of **event**, **condition**, and **action**.
- Can be defined **before** or **after** the triggering event.
- Use row-level triggers for per-row actions and statement-level triggers for batch actions.
- Proper use enhances data integrity and business rules enforcement.
- Use with caution to avoid unintended consequences.